

```

1 %=====
2 % Deterministic Cake-Eating Problem: CRRA Utility
3 %=====
4
5 clear
6 clc
7
8 %Printing the outputs as a text file if ==1
9 output_file=0;
10
11 %LaTeX Interpreter if ==1 (requires LaTeX to be installed if compiled on Octave)
12 latex_interpreter=0;
13
14 %-----
15 % CRRA Utility Function (c.^(1-gamma)-1)/(1-gamma)
16 %-----
17 gamma=2.0; %gamma>0
18 if gamma==1
19     u = @(c) log(c);
20 else
21     u = @(c) (c.^(1-gamma)-1)./(1-gamma);
22 end
23
24 %-----
25 % Grid
26 %-----
27 dimW=5000; %Number of states
28 gridW=linspace(0.01,1,dimW); %Uniform grid over cake size
29
30 %-----
31 % Defining the Auxiliary Matrix of Consumption
32 % for Each Combination of Today and Tomorrow's States
33 %-----
34 % Consumption can be recovered from current and next period states.
35 % Creating the (dimW,dimW)-matrix such that an (i,j) element is
36 % the amount of consumption associated with
37 % w=gridW(i) (current state is the i-th node) and
38 % w_prime=gridW(j) (next period's state is the j-th node).
39 % Note that a choice variable is the next-period state.
40 consumption = gridW'*ones(1,dimW)-ones(dimW,1)*gridW;
41 % gridW'*ones(1,dimW) yields the (dimW,dimW) matrix ( gridW' ... gridW' )
42 % ones(dimW,1)*gridW yields the (dimW,dimW) matrix ( gridW; ... gridW )
43 %
44 util = ( u(consumption(2,1)) - 10^3 ) .* ones(dimW,dimW); %(dimW,dimW)-matrix
45 mask = consumption > 0;
46 util(mask) = u(consumption(mask)); %when u(c)>0, util(c)=u(c)
47 %alternative (slower)
48 %util=zeros(dimW,dimW);
49 %for i=1:dimW
50 % for j=1:dimW
51 % if consumption(i,j)>0
52 %     util(i,j)=u(consumption(i,j));
53 % else
54 %     util(i,j)=u(consumption(2,1))-10^3;
55 % end
56 % end
57 %end
58

```

```

59 %-----
60 % Setting Parametric Values for the Value Function Iteration
61 %-----
62 beta=0.8;           %Discount Factor
63 max_iter=10000;     %The Maximum Number of Iterations
64 toler=0.1^15;       %Tolerance Value
65 converge=0;         %An optional binary variable
66 % converge takes 1 when the value-function iteration ended
67 % within the maximum number of rounds (max_iter).
68
69 %-----
70 % Initialize the Value Function (Initial Guess)
71 %-----
72 % V is a (1, dimW)-matrix with (1,i)-th element denoting the value
73 % (of a candidate value function) associated with state being i (i.e., gridW(i)).
74 % Here, simply start V with the vector of zero.
75 V=zeros(1,dimW);
76
77 %-----
78 % Value function Iteration
79 %-----
80 tic %start stopwatch timer (optional)
81 for i=1:max_iter
82     [newV,index]=max( (util + beta*ones(dimW,1)*V)' );
83     % The (i,j) element of the matrix (u(consumption) + beta*ones(dimW,1)*V)
84     % is the utility associated with state i and choice j
85     % max(A') is a row vector containing the maximum value of each row of A
86     % newV is a (1,dimW)-matrix with (1,i) element denoting
87     % the value of new value function associated with state i
88     % index is a (1,dimW)-matrix with (1,i) element denoting
89     % optimal choice ("optimal j") for each current state i
90
91     %Alternative
92     %[newV,index]=max( (util + beta*ones(dimW,1)*V),[],2 );
93     %newV=newV';
94
95     if norm(newV-V,Inf)<toler %alternative: if max(abs(newV-V))<toler
96         converge=1;
97         break;
98     end %end of if
99     V=newV;
100 end %end of for
101 num_policy=gridW-gridW(index); %Policy function (c=w-w')
102 comp_time=toc; %end stopwatch timer and substitute time into comp_time (optional)
103
104 %-----
105 % Checking Whether the Value Function Converged (Optional)
106 %-----
107 if output_file==1
108     dfile="DP_cake_eating_CRRA.txt";
109     if exist(dfile, 'file')
110         delete(dfile);
111     end
112     diary(dfile)
113     diary on
114 end
115
116 if converge==1

```

```

117     fprintf('The value function iteration converged with %d iterations.\n', i);
118     fprintf('The value function iteration took %f seconds.\n', comp_time);
119 else
120     fprintf('The value function iteration did not converge.\n');
121 end
122
123 if output_file==1
124     diary off
125 end
126
127 %-----
128 % Plotting Policy Function
129 %-----
130 % Plot analytical and numerical solutions to the policy function
131 % Analytical solution:  $h(w)=(1-\beta^{1/\gamma})w$ 
132 exact_policy=(1-beta^(1/gamma))*gridW;
133
134 figure(1)
135 plot(gridW,[exact_policy;num_policy],'LineWidth',1);
136 grid on
137 axis([0 1 0 (1-beta^(1/gamma))*1.05]);
138 if latex_interpreter==1
139     title('Optimal Policy Function','Interpreter','latex','FontSize',14);
140     xlabel('Size of Cake','Interpreter','latex','FontSize',12);
141     ylabel('Consumption','Interpreter','latex','FontSize',12);
142     legend('Analytical','Numerical','Location','southeast','Interpreter','latex');
143 else
144     title('Optimal Policy Function','FontSize',14);
145     xlabel('Size of Cake','FontSize',12);
146     ylabel('Consumption','FontSize',12);
147     legend('Analytical','Numerical','Location','southeast');
148 end
149
150 %-----
151 % Plotting Value function
152 %-----
153 if gamma==1
154     exact_V= ((log(1-beta)/(1-beta))+ (beta/((1-beta)^2))*log(beta))*ones(1,dimW)+ (log(gridW)./(1-
beta));
155 else
156     exact_V= (1/(1-gamma))*((1-beta^(1/gamma))^(-gamma) -(1/(1-beta))) + (1-beta^(1/gamma))^(-
gamma)*u(gridW);
157 end
158 figure(2)
159 plot(gridW, [exact_V;V],'LineWidth',1);
160 grid on
161 xlim([min(gridW) max(gridW)]);
162 if latex_interpreter==1
163     title('Value Function','Interpreter','latex','FontSize',14);
164     xlabel('Size of Cake','Interpreter','latex','FontSize',12);
165     ylabel('Value','Interpreter','latex','FontSize',12);
166     legend('Analytical','Numerical','Location','southeast','Interpreter','latex');
167 else
168     title('Value Function','FontSize',14);
169     xlabel('Size of Cake','FontSize',12);
170     ylabel('Value','FontSize',12);
171     legend('Analytical','Numerical','Location','southeast');
172 end

```

```

173
174 %-----
175 % Computing Optimal Consumption/State Paths
176 %-----
177 num_period=50; %The number of periods for computation
178 % num_period has to be relatively small so that the optimal consumption
179 % is larger than the minimum node.
180 % In the program, once the optimal consumption hits the minimum node then
181 % stays at the minimum node.
182 analytical_W=(beta^(1/gamma)).^(0:num_period-1);
183 analytical_C=(1-beta^(1/gamma))*analytical_W;
184
185 Windex=zeros(1,num_period+1);
186 % Windex is a (1,num_period+1)-matrix with (1,i) element
187 % denoting state at period (i-1) on the optimum path
188 Windex(1,1)=dimW; %At time 0, W_0=dimW-th (initial) state
189 for period=1:num_period
190     if Windex(1,period)>1
191         Windex(1,period+1)=index(Windex(1,period));
192     else
193         Windex(1,period+1)=1;
194     end
195 end
196 Wsim=gridW(Windex);
197 Csim=Wsim(1:num_period)-Wsim(2:num_period+1);
198
199 figure(3)
200 plot(0:num_period-1,[analytical_C;Csim],'LineWidth',1);
201 grid on
202 if latex_interpreter==1
203     xlabel('Period','Interpreter','latex','FontSize',12);
204     ylabel('Consumption','Interpreter','latex','FontSize',12);
205     title('Optimal Consumption Path','Interpreter','latex','FontSize',14);
206     legend('Analytical','Numerical','Interpreter','latex');
207 else
208     xlabel('Period','FontSize',12);
209     ylabel('Consumption','FontSize',12);
210     title('Optimal Consumption Path','FontSize',14);
211     legend('Analytical','Numerical');
212 end
213
214 figure(4)
215 plot(0:num_period-1,[analytical_W;Wsim(1:num_period)],'LineWidth',1);
216 grid on
217 if latex_interpreter==1
218     xlabel('Period','Interpreter','latex','FontSize',12);
219     ylabel('Cake Size','Interpreter','latex','FontSize',12);
220     title('Optimal Path of States','Interpreter','latex','FontSize',14);
221     legend('Analytical','Numerical','Interpreter','latex');
222 else
223     xlabel('Period','FontSize',12);
224     ylabel('Cake Size','FontSize',12);
225     title('Optimal Path of States','FontSize',14);
226     legend('Analytical','Numerical');
227 end
228

```